Phase 1, days 1 to 30: Orient

EY: Talent Data Strategy

C0 Start here: foundations for Fabric

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses

Keep data once, in a typed table format such as Delta, and let each engine read it, instead of exporting copies to untyped files.

CSV, Parquet and Delta	CSV: text, no types. Parquet: columnar, typed, compressed. Delta: Parquet files plus a transaction log with versions.
Lake, warehouse, lakehouse	Lake: files, structure on read. Warehouse: typed tables for SQL. Lakehouse: open tables in a lake, read by Spark and SQL.
ELT and medallion stages	Load raw first, transform inside the platform: bronze (as received), silver (clean), gold (aggregated).
Fabric's names for these	OneLake is the one lake (GA). Lakehouse tables are Delta (GA). The SQL analytics endpoint queries them with T-SQL (GA).

C0-L2 Power BI basics, reports, semantic models and measures

Put ratios in measures that divide totals, and let filter context decide what each total covers.

```
Weighted reach % = DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

Data, semantic model, report	Data sits in source tables. The model describes tables, relationships and measures. Reports hold visuals that query the model.
Calculated column against measure	A column is worked out per row and stored. A measure is worked out when a visual asks, in that visual's filter context.
Ratios in Power BI	A measure dividing totals: DIVIDE (SUM (reach), SUM (base)). Never average per-row percentages.
Import, DirectQuery, Direct Lake	Import copies and refreshes. DirectQuery queries the source each time. Direct Lake (GA) reads OneLake Delta tables directly.

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint

One Delta table, two engines: write and transform with Spark, query with T-SQL on the SQL analytics endpoint.

```python
from pyspark.sql import functions as F

df = spark.read.table("respondents")
totals = df.filter(F.col("final_weight") > 0).groupBy("region").agg(F.sum("final_weight").alias("total"))
totals.show()
```

```sql
SELECT region, SUM(final_weight) AS total
FROM dbo.respondents
WHERE final_weight > 0
GROUP BY region;
```

Two engines over one lakehouse table	Spark in notebooks reads and writes. The SQL analytics endpoint queries with T-SQL and is read-only for table data.

Append rows to a lakehouse table from Spark	df.write.mode("append").saveAsTable("respondents"). mode("overwrite") replaces the rows.
Spark transformations against actions	filter, groupBy, agg, select describe a new DataFrame. show, count, collect run the plan.
Where to put T-SQL writes	In a Warehouse. The lakehouse's SQL analytics endpoint does not write table data.

C1 Fabric architecture

C1-L1 Tenant, capacity, workspace and item

The capacity decides who can view, how much runs at once and what it costs. Separate environments into workspaces, and give production its own capacity.

Fabric hierarchy	Tenant, capacity, workspace, item.
What F64 means	An F SKU with 64 capacity units. F SKUs run from F2 to F2048.
Free-licence viewers of Power BI content	Need an F64 or larger (or P) capacity. Smaller F SKUs need Pro, PPU or trial licences for viewers.
Pausing a capacity	Stops compute billing from then on, bills remaining smoothed and overage usage at once, and stops its items.

C1-L2 OneLake, one lake per tenant

One lake, one copy, many engines. Address data by GUID in anything automated.

```
https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>.<itemtype>/<path>
abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/<path>
```

How many OneLakes per tenant?	Exactly one. You cannot create or delete OneLakes.
OneLake table formats	Delta Parquet or Iceberg, open formats every Fabric engine reads.
OneLake path by name	abfss://<workspace>@onelake.dfs.fabric.microsoft.com/ <item>.<itemtype>/Tables/<table>.
Why GUIDs in automation	They survive renames, avoid special-character problems in abfs URIs, and drop the item type.

C1-L3 Workspace roles, granted to groups

Roles go to groups, write access in production goes to automation, and people read.

Fabric workspace roles	Admin, Member, Contributor, Viewer.
Admin-only rights	Update or delete the workspace, manage admins, connect Git, create a workspace identity.
Viewer and lakehouse data	Reads with T-SQL (ReadData). Not through Spark or OneLake APIs (ReadAll starts at Contributor).
Who writes to production?	An automation identity as Contributor. People get Viewer, through groups.

C2 Lakehouse, Warehouse and SQL analytics endpoint

C2-L1 Lakehouse Tables and Files, and schemas

Raw files in Files, Delta tables in Tables, and schema-qualified names in shared code.

Lakehouse Tables vs Files	Tables: managed Delta tables, visible to SQL and Direct Lake. Files: raw and non-Delta data.
Non-Delta folder under Tables	Shown in the Unidentified area, not queryable.
Lakehouse schemas	On by default for new lakehouses. dbo is the fixed default schema.
Spark four-part name	workspace.lakehouse.schema.table. A bare name means dbo in the default lakehouse.

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

Map the types, declare keys as NOT ENFORCED hints, and check uniqueness yourself after every load.

Primary keys in Fabric Warehouse	Only PRIMARY KEY NONCLUSTERED ... NOT ENFORCED. Duplicates are not blocked.
nvarchar, money, datetime in Fabric Warehouse	Not supported. Use varchar, decimal, datetime2.
MERGE and TRUNCATE in Fabric Warehouse	Both supported. MERGE is generally available.
Triggers in Fabric Warehouse	Not supported. Put the logic in the loading procedure.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

The data and the logic are the same. The item type decides whether the procedure runs at all, and whether a failure leaves the tables consistent.

Lakehouse, Warehouse or endpoint: the deciding question	Who writes the data, and in which language? Spark writes to a lakehouse, T-SQL writes to a warehouse.
What can you do on the SQL analytics endpoint?	SELECT and create views, functions and procedures. No INSERT, UPDATE or DELETE on table data.
When do you need a Warehouse over a Lakehouse?	T-SQL writes, stored procedure loads, and multi-table transactions.
Read a lakehouse table from a warehouse in the same workspace	Use a three-part name: lakehouse.schema.table. No copy.
A new lakehouse table is missing from the endpoint	Metadata sync lag. Usually under a minute. Refresh or retry before dependent steps.

C3 Unifying datasets

C3-L1 OneLake shortcuts, and the delete that goes through

Shortcuts save copies. Treat every path under a shortcut as someone else's data.

What a shortcut is	A folder that points to data in OneLake or external storage, without copying it.
Delete the shortcut vs delete inside it	Deleting the shortcut keeps the data. Deleting files inside it deletes them in the target, if you can write there.
Shortcut as a table	Create it under Tables, pointing at Delta data.
External shortcut caching	OneLake caches read files for 1 to 28 days to cut egress.

C3-L2 Mirroring operational databases into OneLake

Mirroring replicates operational databases for free compute and generous storage, within per-source limits. Reconcile the table list after setup.

Shortcut or mirroring?	Shortcut for open-format data in storage. Mirroring for databases and catalogs, replicated when proprietary.
Mirroring cost	Replication compute is free. Storage is free up to 1 TB per CU. Queries use capacity.
Azure SQL Database mirroring limits	Up to 1,000 tables. Primary keys on datetime2(7), datetimeoffset(7) or time(7) block a table.
After setting up a mirror	Reconcile table lists and row counts with the source.

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

Mirroring Unity Catalog copies no data and no permissions. Decide what is shareable, scope the connection, and rebuild access rules in Fabric.

What mirroring Unity Catalog moves	Only catalog metadata. Data is read in place through shortcuts.
Unity Catalog tables that do not mirror	Tables with row filters or column masks, federated, Delta Sharing and streaming tables, views and materialized views.
Unity Catalog permissions in Fabric	Not carried over. Rebuild access with Fabric's permission model.
Which identity reads mirrored data?	The connection's credential, for all queries. Scope it tightly.

C5 Spark notebooks in Fabric

C5-L1 From Databricks notebooks to Fabric notebooks

Replace dbutils with notebookutils, secrets with Key Vault, inline installs with environments, and pass real timeouts to child notebooks.

Fabric replacement for dbutils	notebookutils (formerly mssparkutils).
Secrets in Fabric notebooks	notebookutils.credentials.getSecret(vault_url, name), from Azure Key Vault.
notebookutils.notebook.run default timeout	90 seconds. Pass timeout_seconds.

%pip in pipeline runs	Turned off by default. Use an environment.

C5-L2 Environments, resource profiles and V-Order

Pin libraries in a Full mode environment, keep the writeHeavy default for ingestion, and turn V-Order on for read-heavy gold tables.

Environment publishing modes	Quick: installs at session start (adds minutes). Full: reproducible snapshot, recommended for pipelines.
V-Order in new workspaces	Off by default (writeHeavy profile). Write cost around 15%, faster reads.
Turn V-Order on for one table	TBLPROPERTIES("delta.parquet.vorder.enabled" = "true").
spark.sql.parquet.vorder.enable	Removed in Runtime 1.3 and later. Use spark.sql.parquet.vorder.default or table and writer options.

C5-L3 Delta tables shared between Databricks and Fabric

Open format, but not every feature for every reader. Design shared gold tables for the weakest reader that needs them.

Delta interoperability rule	Every reader must support every feature the writer put in the table.
TIMESTAMP_NTZ in Fabric	Spark yes. SQL analytics endpoint and Direct Lake no.
Runtime 2.0 default table protocol	Reader 3, writer 7, deletion vectors on. Python notebooks cannot read deletion vectors.
Check a table's features	DESCRIBE DETAIL: minReaderVersion, minWriterVersion, table features.

C10 DevOps and cost

C10-L1 Git integration for Fabric workspaces

One workspace per developer on its own branch, pull requests into main, and a data reload plan, because Git holds definitions, not data.

What Git integration stores for a lakehouse	Metadata only: name, shortcuts, OneLake security roles. Not tables or files.
Commit vs Update	Commit exports workspace changes to Git. Update imports Git changes and overwrites the workspace version.
Who connects a workspace to Git?	Admin only. Members and Contributors commit and update. Viewers see no Git information.
Recommended development process	One workspace per developer on its own branch. Pull requests into main.

C10-L2 Deployment pipelines, dev to test to production

Stage-specific settings belong in deployment rules. Compare before you deploy. Reload data after, because deployments copy items, not data.

Deployment pipeline stages	2 to 10 stages, default Development, Test, Production. One workspace per stage.

What a deployment copies	Item content and metadata. Not data: refresh models and reload tables after.
Deployment rule types	Data source, parameter, default lakehouse. Defined in the target stage on the item.
Deploying over an existing item	Content is overwritten. ID, URL and permissions stay the same.

C10-L3 Capacity, smoothing and throttling

Know which of your jobs are background, run them off-peak, protect production with surge protection, and read the Metrics app before you resize.

Smoothing windows	Interactive: 5 to 64 minutes. Background: 24 hours. Timepoints are 30 seconds, 2,880 a day.
Throttling stages	Up to 10 min: none. 10 to 60: 20 s interactive delay. 60 min to 24 h: interactive rejection. Over 24 h: all rejected.
Surge protection	Capacity level: earlier background rejection. Workspace level: cap a workspace's CU share over 24 hours, except Mission Critical.
Capacity Metrics app	Installed by a capacity admin. Health page for throttling. Compute page for 14 days of usage.

A4 Unity Catalog

A4-L1 The three-level namespace, one catalog per environment

The strong version reads exactly the table its config names, whatever the session's current catalog is.

```
research_prod.research_silver.nat_respondents
```

```
def table_name(catalog: str, layer: str, tracker: str) -> str:
    suffix = {"bronze": "raw", "silver": "respondents", "gold": "metrics"}[layer]
    return f"{catalog}.research_{layer}.{tracker}_{suffix}"
```

Unity Catalog name format	catalog.schema.table, for example research_prod.research_silver.nat_respondents.
What does each level hold in this project?	Catalog: environment. Schema: layer or governance. Table: tracker at a grain.
Why never write a two-part table name in pipeline code?	It resolves against the session's current catalog, which varies by cluster, notebook and job.
Where do table names come from?	One function built from config, such as table_name(catalog, layer, tracker).

A4-L2 Grant to groups, grant the least, grant at the right level

Three rules: groups not users, schemas not tables, SELECT for people and MODIFY for pipelines.

```
GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
  `research_analysts`;
SHOW GRANTS ON SCHEMA research_prod.research_gold;
```

Privileges needed to read a table	USE CATALOG on the catalog, USE SCHEMA on the schema, SELECT on the table or schema.

Why grant SELECT on the schema rather than the table?	Schema grants are inherited by every current and future table in it.
Who gets MODIFY on production silver and gold?	The pipeline service principal. People get SELECT.
Why never grant to a user account?	User grants outlive the reason for them. Group membership is managed in one place.

A4-L3 Row filters and column masks in Unity Catalog

The strong script runs on Databricks, names groups instead of people, and uses three-part names throughout.

```
CREATE FUNCTION research_prod.research_governance.respondent_filter(tracker_id
  STRING)
RETURN is_account_group_member('research_approved_analysts') OR tracker_id = 'TRK-
  PUBLIC';

ALTER TABLE research_prod.research_silver.nat_respondents
SET ROW FILTER research_prod.research_governance.respondent_filter ON (tracker_id);
```

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode SET MASK research_prod.research_governance.mask_postcode;
```

Row filter in Databricks	A SQL UDF returning BOOLEAN, bound with ALTER TABLE t SET ROW FILTER f ON (col).
Column mask in Databricks	A SQL UDF taking the column value, bound with ALTER TABLE t ALTER COLUMN c SET MASK f.
is_account_group_member or is_member?	is_account_group_member: Unity Catalog groups are account-level. is_member checks workspace groups.
What is CREATE ROW ACCESS POLICY?	Snowflake syntax. Databricks rejects it. Reference defect 4.